
donation-service (:4002)

Request flow, file by file

File-by-file request flow

Event-driven microservices · Go · Kafka · Redis

5 — donation-service (:4002)

Job: create and serve donations, and get `donation.created` onto Kafka without ever losing one. The most involved Node service.

```
index.js                                ▶ startup order matters
├─ await connectDb()                    config/db.js      (hard dependency)
├─ await connectProducer()              config/kafka.js  (BEST EFFORT –
│   ▶ a failure is logged, NOT fatal: donations are still
│   accepted and their events retried by the sweeper)
├─ startOutboxSweeper()                 events/outbox.js  every 15s
└─ createApp().listen()

— POST /donations (multipart/form-data) —————
▼
routes/donation.routes.js
├─ requireAuth                         middleware/requireAuth.js
├─ requireRole('DONOR')
├─ uploadImages                       middleware/upload.js (multer → disk)
│   (!) MUST run before validation: with multipart, req.body
│   does not exist until multer has parsed the stream
├─ handleUploadErrors
├─ validateBody(...)                  routes/donation.schemas.js
▼
controllers/donation.controller.js  createDonation()
├─ ① no lat/lng sent?
│   └─ clients/geocodingClient.js → geocoding-service :4003
│       ▶ best effort: a failure leaves the donation
│       un-geocoded rather than rejecting it
├─ ② models/Donation.js  Donation.create({ ..., eventPublished: false })
│   ▶ DURABLE FIRST – this is the outbox
│   ▶ location uses `default: undefined` so an un-geocoded
│   donation has NO location field at all (a Point with
│   no coordinates is rejected by the 2dsphere index)
├─ ③ events/donationEvents.js  publishDonationCreated()
│   └─ config/kafka.js publish() → topic donation.created
│       key = donationId
├─ ④ if published → donation.eventPublished = true; save()
│   (!) a SKIPPED publish must count as failure, or the
│   sweeper never retries it
└─ 201 { donation, assignmentQueued, notice? }
    ▶ 201 even if Kafka was down
```

The outbox sweeper, and the reads

```
— background, every 15s —————  
events/outbox.js  sweepOutbox()  
  └─ Donation.find({ eventPublished: false })  
  └─ publishDonationCreated()    ▶ under the ORIGINAL traceId  
  └─ set the flag on success
```

```
— GET /donations —————  
controllers/donation.controller.js listDonations()  
  ▶ DONOR sees only their own; AGENT sees all  
  ▶ NOTE: donation.status here goes STALE after creation.  
    tracking-service is the authority; the UI reads that.
```

Appendix — the shared skeleton

Repeated here so this page stands alone. Every Node service has these same files doing the same jobs.

Every Node service has these same files doing the same jobs. Once you know this, each service below is just "the skeleton, plus X".

```
src/
├─ index.js           ▶ bootstrap: connect dependencies, THEN listen
├─ app.js             ▶ build the middleware chain (no port binding)
├─ config/
│   ├─ env.js         ▶ read + validate env vars, fail fast at startup
│   ├─ db.js          ▶ mongoose connection
│   └─ redis.js       ▶ redis client (only where Redis is used)
├─ middleware/
│   ├─ requestContext.js ▶ x-trace-id in/out, attaches req.log
│   ├─ requireAuth.js   ▶ verify JWT → req.user; requireRole(...) gate
│   ├─ validate.js      ▶ schema check → 400 with field-level details
│   └─ errorHandler.js ▶ the ONE place every error is formatted
├─ routes/            *.routes.js ▶ URL → middleware chain → controller
│                      *.schemas.js ▶ the validation schemas
├─ controllers/       ▶ the actual work for one endpoint
├─ models/            ▶ mongoose schemas
└─ utils/
    ├─ jwt.js          ▶ signToken / verifyToken
    ├─ ApiError.js     ▶ typed errors (badRequest, notFound, forbidden...)
    └─ logger.js       ▶ structured JSON logging
```

The universal request pipeline



Why `index.js` and `app.js` are separate. `app.js` builds the Express app but never binds a port; `index.js` connects the database *first*, then listens. That split is what lets the tests drive the app in-process with `supertest` without binding a port or racing on one — and it means a service never accepts a request it cannot serve.

Why the error handler is last. Every failure, from any middleware or controller, funnels into one place. That is also where donation-service cleans up orphaned uploads — putting cleanup on the happy path missed every request that failed validation.